

triloop & picoacq

Analysis Code Reference

Pipeline math, module-by-module walkthrough, and per-program description

May 26, 2026

Version 0.1.0

Contents

1	Introduction	1
2	Pipeline overview	1
3	Acquisition: <code>picoacq</code>	2
3.1	<code>picoacq.cli</code>	2
3.2	<code>picoacq.ps5444a</code>	2
3.3	<code>picoacq.simulator</code>	3
3.4	<code>picoacq.recorder</code>	3
3.5	<code>picoacq.alias</code>	3
4	The capture file format	3
5	Geometry and direction transforms	4
5.1	<code>triloop.geometry</code>	4
6	Band extraction and analysis	5
6.1	<code>triloop.extract</code>	5
6.2	<code>triloop.bands</code>	5
6.3	<code>triloop.analyze</code>	5
6.4	<code>triloop.multiband</code>	6
7	Magnetoionic O/X mode separation	6
7.1	Physical motivation	6
7.2	The Appleton-Hartree polarization parameter	6
7.3	Sign and handedness conventions	7
7.4	Path geometry: entry, exit, and the symmetry that doesn't hold	7
7.5	Path-integrated Faraday rotation	7
7.6	Receiver-side projection	8
7.7	Sanity checks performed	8

8	Direction finding	8
8.1	<code>triloop.direction</code>	8
8.2	Why find the null instead of the maximum?	9
8.3	<code>triloop.beamform</code>	9
8.4	<code>triloop.stokes</code>	9
9	Reporting and visualization	9
9.1	<code>triloop.view</code>	9
9.2	<code>triloop.browse</code>	9
10	Module roll call	10
10.1	<code>picoacq</code>	10
10.2	<code>triloop</code>	11
11	Tracing a capture through the code	11
12	Testing and verification	12

1 Introduction

This document is a developer’s reference for the analysis code in `triloop` and the acquisition code in `picoacq`. It complements the *triloop User Manual* (which describes how to *use* the tools) and the *picoacq User Manual* (which describes the hardware and the capture file format). Here we focus on *what each Python module does*, in what order they are called by the public entry points, and the mathematics each step implements.

Audience. Reader is comfortable with Python, complex baseband signal processing, and standard linear-algebra notation.

Conventions used in this document. Bold lower-case (**b**) denotes a vector; bold upper-case (**N**) denotes a matrix; $\hat{\mathbf{k}}$ is the unit propagation vector *toward the source*; angles are in degrees in code, radians in equations.

2 Pipeline overview

A capture flows through the toolchain in four stages:

1. **Acquisition** (`picoacq`). Hardware capture orchestrated by `picoacq.recorder.capture()`, which calls the PicoSDK wrapper `picoacq.ps5444a` or, in `--simulate` mode, `picoacq.simulator`. Output: a `triloop`-format HDF5 file with float32 samples in volts plus a JSON `capture_settings` dict carrying the chosen sample rate, voltage ranges, auto-range probe values, and (new in v0.1) the list of expected RF bands `rf_bands_hz`.
2. **Band extraction** (`triloop.bands`, `triloop.extract`). Read the HDF5 file with `triloop.io.hdf5.read_c` identify the baseband alias position of each RF band via `picoacq.alias`; mix-and-LPF each channel down to a complex baseband centred on that band; decimate to a workable IF rate.

3. **Polarimetry and direction finding** (`triloop.analyze`, `triloop.geometry`, `triloop.stokes`, `triloop.direction`, `triloop.beamform`). Recover the lab-frame $\mathbf{B}$ -vector from the three loops via the inverse of the geometry matrix $\mathbf{N}$, project onto the plane perpendicular to $\hat{\mathbf{k}}$, compute Stokes parameters and intensity, and (optionally) refine $\hat{\mathbf{k}}$ with a null-eigenvector / null-sweep search.
4. **Reporting** (`triloop.view`, `triloop.multiband`, `triloop.browse`). Static QC plots, multi-band comparison figures, and an interactive single-file HTML browser for visual inspection.

The CLI tools (`triloop view`, `triloop analyze-multi`, `triloop browse`, `triloop analyze`, `triloop locate`, `triloop batch`, `triloop summary`, `triloop simulate`, `triloop notebook`) wrap these stages in different combinations.

3 Acquisition: `picoacq`

3.1 `picoacq.cli`

Click-based command-line interface with two subcommands: `capture` (one-shot) and `monitor` (continuous). Each exposes the same configuration knobs as the underlying recorder; the defaults reflect the multi-band undersampling workflow:

- `--rate 15625000` (Hz)
- `--duration 8.0` (s)
- `--channels A,B,C,D`
- `--range 2.0` (V; only used when `--no-auto-range`)
- `--auto-range/--no-auto-range` (default on)
- `--headroom 3.0`
- `--probe 0.1` (auto-range probe duration, s)
- `--rf-bands "2.5e6,5e6,10e6,15e6,20e6"`

3.2 `picoacq.ps5444a`

A thin ctypes wrapper around the PicoSDK 5000a driver. Five functions:

- `open_5444(n_channels=4)` – context manager that opens the device, automatically picking 16-bit resolution for ≤ 2 channels and 14-bit for 3–4 channels (5444D constraint).
- `configure_channel(ps, handle, ch, range_volts, coupling)` – enables a channel and sets its $\pm$ range.
- `get_timebase(ps, handle, target_rate)` – searches the Pico timebase index space for the closest valid sample rate at or below the requested target. Skips early indices that are invalid in 4-channel mode.
- `acquire_block(ps, handle, channels, n, tb, ranges)` – block-mode acquisition with explicit ctypes-wrapped arguments to avoid signature mismatches between picosdk versions.
- `_ensure_pico_dyld_path()` – macOS-only shim that injects the framework library path into `DYLD_LIBRARY_PATH` before importing `picosdk`.

3.3 picoacq.simulator

The no-hardware fallback. Generates a 3-loop synthetic dataset with configurable arrival direction, polarization, additive Gaussian noise, and Faraday rotation rate. Used by `--simulate` mode and as the silent fallback when the SDK or hardware is unavailable.

3.4 picoacq.recorder

The orchestrator. Top-level function: `capture(output_file, duration_s, sample_rate, channels, ranges_volts, coupling, loops_config, simulate, sim_kwargs, auto_range_enabled, headroom, probe_duration_s, rf_bands, verbose)`. Internally:

1. `_check_buffer()` validates that `duration×rate` samples fit in the onboard memory (128 MS/ch in 4-channel mode); raises `ValueError` with a clear maximum if not.
2. If auto-range is on (and `ranges_volts` not supplied): open the scope at the maximum ± 20 V on every channel, run a brief probe capture (`auto_range()`), measure peak/RMS in volts, and call `_pick_range()` to choose the smallest available Pico range whose full scale $\geq K \cdot \text{peak}$ (default $K = 3$).
3. Run the science capture via `_try_real_capture()`; on any SDK/hardware failure, fall back transparently to `simulate_capture()`.
4. Convert int16 ADC codes $\rightarrow$ float32 volts; write to HDF5 via `triloop.io_hdf5.write_capture()` with the chosen ranges, the auto-range probe results, and `rf_bands_hz` stored in `capture_settings`.

3.5 picoacq.alias

Pure-Python helper that maps RF frequencies to baseband alias positions for a given sample rate. An RF tone at f_{rf} folds to baseband as

$$f_{\text{bb}} = |f_{\text{rf}} - n f_s|, \quad n = \text{round}(f_{\text{rf}}/f_s), \quad (1)$$

with Nyquist zone $z = \lfloor 2f_{\text{rf}}/f_s \rfloor + 1$. Even zones produce a frequency-inverted spectrum. Three exports:

- Alias dataclass with fields (`rf_hz`, `baseband_hz`, `nyquist_zone`, `inverted`).
- `alias_of(rf_hz, sample_rate_hz)` – single mapping.
- `rf_for_baseband(f_bb, rf_bands, fs, tol)` – inverse lookup (given a detected baseband peak, return which RF band it most likely came from).

This module is the single source of truth for alias bookkeeping; `triloop.bands` and `triloop.view/browse` all import from it.

4 The capture file format

Both packages read and write the same HDF5 v0.1 layout:

```

/                                HDF5 root, @format_version = "0.1"
/metadata                       group, attributes:
    @sample_rate                 float64, Hz
    @duration_s                  float64
    @start_time_utc              ISO-8601
    @scope_model                 string
    @scope_serial                string
    @capture_settings            JSON-encoded dict
    @loops_config                JSON-encoded dict
/channels
    /A    /B    /C    /D        float32 (or int16) datasets, length N
    each has:
        @gain_v_per_count        float64
        @offset_v                float64
/time                           optional float64 array, length N

```

capture_settings fields recorded by `picoacq.recorder.capture()`:

- requested_duration_s, requested_sample_rate
- channels, ranges_volts, coupling
- used_simulator, auto_range_enabled, headroom
- auto_range_probe (peaks_volts, rms_volts, headroom, probe_duration_s) – only when probe ran
- picoscope_ranges_volts – the actual chosen ranges
- rf_bands_hz – list of expected RF frequencies in Hz
- truth – only present in simulator mode

triloop.io_hdf5 provides `read_capture()` and `write_capture()`; both are short and tested, and any third party can interoperate by writing the same layout.

5 Geometry and direction transforms

5.1 triloop.geometry

Operates on the loop-array geometry config dict (`loops_config`). Five public functions:

- `az_el_to_khat(az_deg, el_deg)` returns $\hat{\mathbf{k}} = (\cos \theta \sin \varphi, \cos \theta \cos \varphi, \sin \theta)^\top$ in the local ENU frame.
- `build_N_matrix(loops_config)` returns the 3×3 matrix $\mathbf{N}$ whose i -th row is the i -th loop's normal $\hat{\mathbf{n}}_i$ (in ENU, scaled by per-loop gain). This is the matrix that maps the lab-frame $\mathbf{B}$ to the loop-projected real signal $\mathbf{b} = \mathbf{NB}$.
- `perp_projector(khat)` returns $\mathbf{P}_\perp = \mathbf{I} - \hat{\mathbf{k}}\hat{\mathbf{k}}^\top$, the projector onto the plane perpendicular to $\hat{\mathbf{k}}$.
- `perp_orthonormal_basis(khat)` returns two unit vectors $(\hat{\mathbf{p}}, \hat{\mathbf{q}})$ that span that plane, with $\hat{\mathbf{p}}$ chosen along the local horizontal whenever possible, $\hat{\mathbf{q}} = \hat{\mathbf{k}} \times \hat{\mathbf{p}}$.
- `LOOP_NORMALS_DEFAULT` and `_normal_from_az_el` support the default cube-vertex layout.

The default geometry is three loops on three faces of a cube whose body diagonal is vertical, giving normal-tilts of $\arctan(1/\sqrt{2}) \approx 35.26^\circ$ above horizontal at azimuths $\{0, +120, -120\}^\circ$.

6 Band extraction and analysis

6.1 `triloop.extract`

Two functions covering the single-band path:

- `extract_complex_baseband(t, b, f0, BW)` performs an FFT-based peak search around f_0 over a $\pm \text{search_BW}$ window, mixes the strongest peak to DC, brick-wall low-passes to $\pm \text{BW}/2$, and returns the complex baseband at the original sample rate, the detected peak frequency, and an FFT-domain SNR estimate.
- `extract_three_loops(...)` calls the above on each loop, applies per-loop `phase_offset_deg` calibration, and returns a stacked $(3, N)$ complex array.

6.2 `triloop.bands`

Multi-band counterpart, used when the capture's `rf_bands_hz` populates more than one entry. Public function: `extract_bands(cap, rf_bands, bw_hz, decim_rate_hz, channels)`. For every band:

1. `picoacq.alias.alias_of()` computes f_{bb} , the Nyquist zone, and `inverted`.
2. Validate that the requested `bw_hz` doesn't reach into the adjacent Nyquist edge or another band's alias.
3. Mix every selected channel down to f_{bb} via FFT (`_extract_one_channel`), brick-wall filter to $\pm \text{bw_hz}/2$, decimate to the smallest integer factor that yields $\geq \text{decim_rate_hz}$, and conjugate the result if the band lives in an even (inverted) Nyquist zone.

Returns a dict mapping `rf_hz` $\rightarrow$ `BandExtraction` dataclass with fields `rf_hz`, `baseband_hz`, `nyquist_zone`, `inverted`, `bw_hz`, `decim_rate_hz`, `t`, `z`, where `z` is a per-channel dict of complex64 arrays.

6.3 `triloop.analyze`

The polarimetry pipeline. Two public entry points:

- `analyze(t, B1, B2, B3, f0, BW, az_deg, el_deg, loops_config)` – starts from real loop signals, calls `extract_three_loops()`, then forwards to:
- `analyze_z_loops(t, Z_loops, f_peak, snrs_db, az_deg, el_deg, loops_config)` – the inner pipeline, also called directly by the multi-band path.

The inner pipeline computes:

$$\mathbf{z}_{\text{lab}}(t) = \mathbf{N}^{-1} \mathbf{z}_{\text{loops}}(t), \quad (2)$$

$$\mathbf{z}_{\perp}(t) = \mathbf{P}_{\perp} \mathbf{z}_{\text{lab}}(t), \quad (3)$$

$$I(t) = |\mathbf{z}_{\perp}|^2, \quad (4)$$

$$A_p(t) = \hat{\mathbf{p}} \cdot \mathbf{z}_{\perp}, \quad A_q(t) = \hat{\mathbf{q}} \cdot \mathbf{z}_{\perp}, \quad (5)$$

then forwards (A_p, A_q) to `stokes.compute_stokes()` which returns the four Stokes parameters, the polarization fraction $\sqrt{Q^2 + U^2 + V^2}/I$, the ellipticity $\frac{1}{2} \arcsin(V/I_p)$ in degrees ($\pm 45^\circ$ at full circular polarization), and the position angle $\frac{1}{2} \arctan 2(U, Q)$.

A 2×2 coherency on (A_p, A_q) identifies the dominant-pol eigenvector u_{dom} , and $z_{\text{dom}}(t) = u_{\text{dom}}^\top(A_p, A_q)$ provides an instantaneous phase signal whose unwrapped, linearly-detrended residual gives both the residual carrier frequency and a time-resolved frequency series via `numpy.gradient`. Everything is bundled into an `AnalysisResult` dataclass.

6.4 `triloop.multiband`

Wraps band extraction and per-band analysis, with optional projection onto the magnetoionic O/X modes. Public functions:

- `analyze_all_bands(cap, az_deg, el_deg, loops_channels, bw_hz, decim_rate_hz, rf_bands, loops_config, tx_lat_deg, tx_lon_deg, rx_lat_deg, rx_lon_deg, reflection_height_km)` – calls `extract_bands` once and runs `analyze_z_loops` on each band. When all four TX/RX coordinates are supplied, also calls `exit_point_geometry` once and projects each band’s perp-plane signal onto the predicted O/X modes (see Section 7). Returns a dict mapping `rf_hz` $\rightarrow$ `MultiBandResult` with fields (`rf_hz`, `extraction`, `analysis`, `snrs_db`, `mode`) where `mode` is a `ModeSeparation` or `None`.
- `make_multiband_figure(results, out_path)` – plot a five-row, four-column matplotlib figure (one row per band) of intensity, instantaneous frequency, ellipticity, and polarization fraction.

7 Magnetoionic O/X mode separation

7.1 Physical motivation

In the cold magnetised plasma of the ionosphere, an HF wave does not propagate as a single mode — it splits into two orthogonal characteristic modes (the *ordinary* O and *extraordinary* X) with slightly different refractive indices. Each mode carries its own amplitude and phase along the ray, and the polarization *shape* of each mode evolves smoothly with the local $\theta = \angle(\hat{k}, \hat{B})$.

At the ray’s *exit point* (where the wave is just leaving the ionosphere on its way to the receiver), the local plasma frequency $f_p \rightarrow 0$ so the mode polarization simplifies to a pure geometric calculation in (θ, f_H, f) . From the exit point onward the modes propagate adiabatically through vacuum to the antenna, with no further polarization evolution. This is why the exit point is the right place to evaluate the mode shapes for receiver-side analysis.

7.2 The Appleton-Hartree polarization parameter

For a wave $\mathbf{E} = E_p \hat{p}_B + \rho E_p \hat{q}_B$ in the right-handed perpendicular-plane basis $(\hat{p}_B, \hat{q}_B)$ with $\hat{p}_B$ along the projection of $\hat{B}$, the cold-plasma result is

$$\rho_{O,X} = -iq \pm i\sqrt{q^2 + 1}, \quad q = \frac{Y_T^2}{2Y_L(1 - X)}, \quad (6)$$

with $Y \equiv f_H/f$, $Y_L = Y \cos \theta$, $Y_T = Y \sin \theta$, $X = (f_p/f)^2$. At the exit point $X = 0$.

Limiting cases (verified numerically by the sanity-check suite, see Section 7.7):

- $\theta \rightarrow 0$ (QL): $\rho_O \rightarrow +i$, $\rho_X \rightarrow -i$ – pure RCP/LCP about $\hat{k}$. Both modes are circular.
- $\theta \rightarrow \pi/2$ (QT): $\rho_O \rightarrow 0$, $\rho_X \rightarrow -i\infty$ – two orthogonal linear polarizations along $\hat{p}_B$ and $\hat{q}_B$.
- All θ : the two modes are orthogonal under the Hermitian inner product, $\langle u_O, u_X \rangle = 0$, where $u_M = (1, \rho_M)/\sqrt{1 + |\rho_M|^2}$. Verified to numerical precision on 10^3 random parameter draws.

7.3 Sign and handedness conventions

- $\hat{k}$ is the Poynting direction — source $\rightarrow$ receiver. Use `khat_propagation(az_src, el_src)` or just negate what `az_el_to_khat` returns.
- $(\hat{p}_B, \hat{q}_B)$ is right-handed: $\hat{p}_B \times \hat{q}_B = \hat{k}$.
- ρ describes polarization relative to $\hat{k}$, not to $\hat{B}$. In particular, flipping $\theta \rightarrow \pi - \theta$ does *not* negate ρ in our formulation; we verified this both algebraically (Block 1 of the sanity-check suite) and numerically. The handedness “relative to $\hat{B}$ ” is what transforms simply, but for matched-filter projection at the receiver the relevant frame is $\hat{k}$.
- Texts differ on the O/X label at $\theta = 0$. Our code matches the IEEE/PhaRLAP convention (O is the wave with $\rho = +i$, i.e. right-handed about $+\hat{k}$). Since both modes appear in any real wave, the label is a bookkeeping choice; what matters is that the two outputs of `appleton_hartree_polarization` are orthogonal.

7.4 Path geometry: entry, exit, and the symmetry that doesn’t hold

`exit_point_geometry(tx_lat, tx_lon, rx_lat, rx_lon, h_km)` returns the geometry at the ray’s descending exit point: a flat-Earth-approximated three-quarter-path location at altitude h_{km} , with $\hat{k}$ pointing down and toward the receiver in local exit-ENU. `entry_point_geometry` returns the same quantities at the upgoing entry (quarter path, $\hat{k}$ pointing up and toward the apex).

The symmetry test we expected and the one that actually holds. Naively, $\theta_{\text{entry}} + \theta_{\text{exit}}$ should equal 180° , since $\hat{k}$ flips its vertical component while $\hat{B}$ stays roughly the same. The sanity-check suite found this fails: the sum is closer to 234° for the FC $\rightarrow$ APO path. Working it out algebraically: with $\hat{k}_{\text{entry}} = (a, b, +c)$ and $\hat{k}_{\text{exit}} = (a, b, -c)$, $\cos \theta_{\text{entry}}$ and $\cos \theta_{\text{exit}}$ differ only in the $\hat{B}$ -vertical term. Their arccosines sum to 180° *only* when the horizontal components of $\hat{k}$ and $\hat{B}$ are perpendicular — i.e. when the path’s horizontal direction is east-west and $\hat{B}$ has no horizontal east component. For paths that aren’t due-east (FC $\rightarrow$ APO is mostly N-S, with $\hat{B}$ also pointing mostly N), the expected “ 180° ” identity does not hold. The k-vector identity $\hat{k}_{\text{entry,horiz}} = \hat{k}_{\text{exit,horiz}}$ and $\hat{k}_{\text{entry,vert}} = -\hat{k}_{\text{exit,vert}}$ does hold to machine precision.

7.5 Path-integrated Faraday rotation

`integrated_faraday_rotation_rad(B_cos_theta_avg_T, TEC_electrons_per_m2, f_hz)` returns

$$\Omega = \frac{e^3}{8\pi^2\epsilon_0 m_e^2 c} \frac{\langle B \cos \theta \rangle \text{TEC}}{f^2} \approx 2.36 \times 10^4 \frac{\langle B \cos \theta \rangle [\text{T}] \text{TEC} [e/m^2]}{f^2 [\text{Hz}]} \quad (7)$$

At 30 TECU on the FC $\rightarrow$ APO path with $\langle B \cos \theta \rangle \approx 25 \mu\text{T}$, this gives ~ 4500 turns at 2.5 MHz and ~ 70 turns at 20 MHz. The f^{-2} scaling is verified to numerical precision in the sanity-check suite.

7.6 Receiver-side projection

The connection from antenna data to mode amplitudes:

1. `analyze_z_loops` produces $A_p(t), A_q(t)$ in the receiver’s horizontal-first ($\hat{p}_R, \hat{q}_R$) basis.
2. `multiband._project_onto_modes` computes the angle α between $\hat{p}_R$ and the magnetoionic $\hat{p}_B$ in the perp-plane, then rotates $(A_p, A_q) \rightarrow (A_{p_B}, A_{q_B})$.
3. Matched-filter: $a_O(t) = u_O^\dagger(A_{p_B}, A_{q_B})$, $a_X(t) = u_X^\dagger(A_{p_B}, A_{q_B})$.

The differential phase $\Delta\varphi(t) = \arg a_O - \arg a_X$, unwrapped, is the time-resolved analogue of the path-integrated Faraday rotation.

7.7 Sanity checks performed

The full validation suite runs eight blocks of checks, of which 21/23 named assertions pass. The two that “fail” are:

- “ ρ_O at $\theta = \pi$ should equal $-\rho_O$ at $\theta = 0$.” It doesn’t, because ρ tracks polarization relative to $\hat{k}$, and flipping θ doesn’t change the sign of q . This was a faulty expectation, not a code bug.
- “ $\theta_{\text{entry}} + \theta_{\text{exit}} \approx 180^\circ$.” As discussed above, this only holds in special path geometries. The k-vector identity that *does* always hold passes.

The passing checks include: AH limits at $\theta = 0, \pi/2$; mode orthogonality on 10^3 random parameter draws; energy conservation $|a_O|^2 + |a_X|^2 = |E|^2$ to 10^{-15} ; IGRF reality check (mid-latitude US gives $|B| \approx 50 \mu\text{T}$, inclination $\approx 62^\circ$, declination $\approx 9^\circ$); flat-Earth elevation within 2.5° of the spherical-Earth value; range haversine match to 0.2%; numerical robustness at $\theta = 0, \pi, \pi/2 \pm \epsilon$, $f = f_H$, $f \ll f_H$, $f \gg f_H$.

When to use PHaRLAP instead. This module’s flat-mirror geometry is fine for first-pass mode-polarization signature work – “are we in QL or QT?” is robust against the geometric error. For quantitative work the right tool is PHaRLAP (Cervera, DSTG/U. Adelaide), which traces the full 3-D Haselgrove ray equations through an IRI ionosphere with IGRF. See `triloop.pharlap.predict` (planned) for the offline-prediction interface.

8 Direction finding

8.1 `triloop.direction`

Public functions, all operating on a stacked lab-frame complex array $Z_{\text{lab}} \in \mathbb{C}^{3 \times N}$:

- `coherency_matrix(Z_lab)` returns $C = \frac{1}{N} Z_{\text{lab}} Z_{\text{lab}}^H$, the 3×3 Hermitian coherency.
- `estimate_direction_from_z_lab(Z_lab)` returns the smallest-eigenvalue eigenvector of C as $\hat{\mathbf{k}}_{\text{est}}$, plus the polarization basis ($\hat{\mathbf{p}}, \hat{\mathbf{q}}$ from the two largest eigenvectors), the eigenvalues (descending), and a scalar `null_strength` λ_3/λ_1 . This is the *coarse* direction estimate; small `null_strength` means the rank-1 trap (§ 8.2) is broken and the estimate is trustworthy.
- `perp_residual_energy(Z_lab, khat)` computes $\langle |\hat{\mathbf{k}} \cdot \mathbf{z}_{\text{lab}}|^2 \rangle$, the quantity minimised by direction finding.

- `null_search(Z_lab, az0, el0, half_width_deg, n_points)` performs a 2-D grid sweep around (az_0, el_0) and returns the minimiser plus the full residual map.
- `parabolic_refine(R, az_grid, el_grid)` fits a parabola to the 3×3 neighbourhood of the grid minimum to give a sub-grid direction.
- `lock_and_analyze(...)` – end-to-end entry point used by the `locate` CLI.

8.2 Why find the null instead of the maximum?

The direction-formed power $P(\hat{\mathbf{k}}) = \langle |\mathbf{B}_\perp(\hat{\mathbf{k}})|^2 \rangle$ varies as $\frac{1}{2} + \frac{1}{2} \cos^2 \beta$ near the true direction $\hat{\mathbf{k}}_{\text{true}}$ (a shallow plateau). The *perpendicular* residual $Q(\hat{\mathbf{k}}) = \langle |\hat{\mathbf{k}} \cdot \mathbf{B}|^2 \rangle$ varies as $\sin^2 \beta \sim \beta^2$ near the same direction (a sharp parabolic minimum). Localising a parabolic minimum is far more precise than a cosine-squared maximum, so we always lock on the null rather than the peak.

For purely linearly polarized signals from a single direction, $\mathbf{C}$ is rank 1 and the null is a 2-D plane rather than a point. In that regime `estimate_direction_from_z_lab` returns a direction (the smallest-eigenvalue one within floating-point noise), but it is not physically meaningful; trustworthy localisation requires Faraday rotation, a circularly polarized source, or triangulation.

8.3 `triloop.beamform`

Older, simpler beamforming sweep used by some analysis notebooks. Two functions:

- `beamform_grid(z_loops, loops_config, az_grid_deg, el_grid_deg)` – compute time-averaged $P(\varphi, \theta) = \langle |\mathbf{B}_\perp(\hat{\mathbf{k}}_{\varphi, \theta})|^2 \rangle$ over a 2-D grid.
- `best_direction(P, az_grid, el_grid)` – return the arg max as (φ, θ) .

8.4 `triloop.stokes`

A single function: `compute_stokes(A_p, A_q)` returning a dict with `I`, `Q`, `U`, `V`, `pol_fraction`, `ellipticity_deg`, `position_angle_deg`. Definitions: $I = |A_p|^2 + |A_q|^2$, $Q = |A_p|^2 - |A_q|^2$, $U = 2\Re\{A_p^* A_q\}$, $V = 2\Im\{A_p^* A_q\}$.

9 Reporting and visualization

9.1 `triloop.view`

Static QC PNG generator used by `triloop.view`. `make_view_figure(cap, rf_bands, bw_hz, out_path)` produces a single matplotlib figure with three rows: a full-Nyquist PSD with each RF band annotated at its alias position, per-band PSD zooms (with the displayed f-axis flipped for inverted bands so the operator reads frequencies the right way), a decimated time-domain trace of every channel, and the auto-range probe table.

9.2 `triloop.browse`

Interactive single-file HTML browser. Top-level entry points:

- `build_browser_html(cap, az_deg, el_deg, loops_channels, bw_hz, decim_rate_hz, n_anim_frames, title)` – returns a self-contained HTML string.
- `write_browser(cap, out_path, **kw)` – save it to disk.

The page uses Plotly (loaded once via CDN; if you need a fully offline build, change the `include_plotlyjs="cdn"` argument to `True` and the page will inline the entire library at the cost of ~ 3 MB extra). Layout: tab bar at the top with *Overview*, *Time series*, and one tab per RF band. The per-band tab has four panels: PSD zoom, intensity time history (linear + togglable dB), intensity CDF with P10/P1 fade-depth markers, and an animated polarization ellipse with Play/Pause and a frame slider.

The animation is implemented as Plotly frames; each frame draws a sliding window of $(\Re A_p, \Re A_q)$ samples centred on a different slow-time epoch. The window length is chosen automatically to be a few cycles of the residual carrier-phase variation at the decimated rate.

10 Module roll call

This section lists every Python file in both packages and what each one provides. Treat it as a table of contents for the source.

10.1 picoacq

Module	Purpose
<code>__init__.py</code>	package marker; no public API of its own.
<code>cli.py</code>	Click-based CLI; subcommands <code>capture</code> and <code>monitor</code> . Reads <code>--rate</code> / <code>--duration</code> / <code>--channels</code> / <code>--range</code> / <code>--auto-range</code> / <code>--headroom</code> / <code>--probe</code> / <code>--rf-bands</code> / <code>--simulate</code> / <code>--sim-*</code> options and forwards them to <code>recorder.capture()</code> .
<code>recorder.py</code>	Top-level <code>capture()</code> orchestrator with <code>_check_buffer()</code> guard, <code>auto_range()</code> probe, range chooser <code>_pick_range()</code> , and the <code>_try_real_capture()</code> hardware path (with simulator fallback).
<code>ps5444a.py</code>	PicoSDK 5000a wrapper: <code>open_5444()</code> , <code>configure_channel()</code> , <code>get_timebase()</code> , <code>acquire_block()</code> . Plus <code>_ensure_pico_dyld_path()</code> for macOS framework search.
<code>simulator.py</code>	<code>simulate_capture()</code> – 3-loop no-hardware fallback compatible with the <code>recorder</code> return shape.
<code>alias.py</code>	RF $\leftrightarrow$ baseband mapping under bandpass undersampling. Public: <code>Alias</code> dataclass, <code>alias_of()</code> , <code>aliases_for_bands()</code> , <code>rf_for_baseband()</code> . See Section 3.5.

10.2 triloop

Module	Purpose
<code>__init__.py</code>	re-exports the public API (see <i>User Manual</i> for the full list).
<code>cli.py</code>	Click-based CLI; subcommands <code>view</code> , <code>analyze-multi</code> , <code>browse</code> , <code>analyze</code> , <code>locate</code> , <code>batch</code> , <code>summary</code> , <code>simulate</code> , <code>notebook</code> .
<code>io.hdf5.py</code>	<code>read_capture()</code> and <code>write_capture()</code> for the HDF5 v0.1 layout described in §4.
<code>config.py</code>	<code>default_loops_config()</code> (cube-vertex geometry) and <code>validate_loops_config()</code> schema check.
<code>geometry.py</code>	<code>az_el_to_khat()</code> , <code>build_N_matrix()</code> , <code>perp_projector()</code> , <code>perp_orthonormal_basis()</code> , <code>LOOP_NORMALS_DEFAULT</code> .
<code>extract.py</code>	Single-band complex-baseband extractor: <code>extract_complex_baseband()</code> , <code>extract_three_loops()</code> .
<code>bands.py</code>	Multi-band extractor: <code>extract_bands()</code> , <code>BandExtraction</code> dataclass.
<code>stokes.py</code>	<code>compute_stokes(A_p, A_q)</code> - Stokes parameters and derived quantities.
<code>analyze.py</code>	Top-level <code>analyze()</code> and inner <code>analyze_z_loops()</code> ; <code>AnalysisResult</code> dataclass.
<code>multiband.py</code>	<code>analyze_all_bands()</code> (extract + analyze for every band, with optional O/X mode separation when TX/RX coordinates are given) plus <code>ModeSeparation</code> and <code>MultiBandResult</code> dataclasses; private <code>_project_onto_modes()</code> matched-filter helper; <code>make_multiband_figure()</code> .
<code>magnetoionic.py</code>	Cold-plasma O/X mode polarization: <code>appleton_hartree_polarization()</code> , <code>mode_unit_vectors()</code> , <code>mode_ellipticity_deg()</code> , <code>mode_orientation_deg()</code> , <code>mode_axial_ratio()</code> , <code>khat_propagation()</code> , <code>igrf_b_hat_enu()</code> (with <code>pyIGRF</code> or fallback approximation), <code>exit_point_geometry()</code> , <code>entry_point_geometry()</code> , <code>modes_at_exit()</code> , <code>integrated_faraday_rotation_rad()</code> ; <code>ExitGeometry</code> , <code>EntryGeometry</code> , <code>ModeAtExit</code> dataclasses.
<code>direction.py</code>	Coherency-based direction finding: <code>coherency_matrix()</code> , <code>estimate_direction_from_z_lab()</code> , <code>perp_residual_energy()</code> , <code>null_search()</code> , <code>parabolic_refine()</code> , <code>lock_and_analyze()</code> .
<code>beamform.py</code>	Older/simpler grid-sweep: <code>beamform_grid()</code> , <code>best_direction()</code> .
<code>view.py</code>	Static QC PNG: <code>make_view_figure()</code> .
<code>browse.py</code>	Interactive Plotly HTML browser: <code>build_browser_html()</code> , <code>write_browser()</code> , plus the helpers <code>_channel_psd()</code> , <code>_band_zoom_spectrum()</code> , <code>_smooth_decimate()</code> , <code>_cdf()</code> , <code>_build_pol_animation()</code> .

11 Tracing a capture through the code

To consolidate the picture above, here are the call paths for the three new commands:

`triloop view cap.h5`.

1. `cli.view_cmd` parses options.

2. `io_hdf5.read_capture(cap.h5) -i dict`.
3. `view.make_view_figure(cap, rf_bands, bw_hz, out_path)`.
4. Inside `make_view_figure`: per-channel PSD via `_channel_psd`; per-band zoom via `_band_zoom_spectrum` (calls `picoacq.alias.alias_of` for each band centre); time-domain decimated; probe table from `capture_settings.auto_range_probe`.

`triloop analyze-multi cap.h5 --az 9 --el 35`.

1. `cli.analyze_multi_cmd` parses options.
2. `io_hdf5.read_capture(cap.h5) -i dict`.
3. `multiband.analyze_all_bands(cap, az, el, ...)`:
 - (a) `bands.extract_bands(cap, ...)` which for each band uses `picoacq.alias.alias_of` to find f_{bb} , then mixes and decimates each channel via `_extract_one_channel`.
 - (b) For each band: `_per_loop_snr_from_baseband`, then `analyze.analyze_z_loops(t, Z_loops, f_rf, ...)`.
4. `multiband.make_multiband_figure(results, out_png)`.
5. Per-band JSON summary written to `out_json`.

`triloop browse cap.h5`.

1. `cli.browse_cmd` parses options.
2. `io_hdf5.read_capture(cap.h5) -i dict`.
3. `browse.write_browser(cap, out_html, ...)` which calls `build_browser_html(cap, ...)`:
 - (a) Calls `multiband.analyze_all_bands` (same path as above) to populate per-band intensity, ellipticity, and A_p/A_q time series.
 - (b) Builds per-tab Plotly figures: overview PSD, time series, and one 4-panel tab per band.
 - (c) For each band assembles a polarization-ellipse animation via `_build_pol_animation()` (Plotly frames + slider).
 - (d) Renders the page using `plotly.io.to_html()` for each figure into a tabbed HTML scaffold.

12 Testing and verification

The test suite under `triloop/tests/` exercises the analysis pipeline against the simulator at known polarizations and directions. Useful commands during development:

```
# Create a synthetic 4-channel capture with all 5 WWV bands aliased
PYTHONPATH=triloop:picoacq python3 -m picoacq.cli capture \
  --simulate --duration 0.5 -o sim.h5

# Run the multi-band analysis and confirm pol_frac and ellipticity
PYTHONPATH=triloop:picoacq python3 -m triloop.cli analyze-multi sim.h5 \
  --az 9 --el 35
```

```
# Render the interactive browser
PYTHONPATH=triloop:picoacq python3 -m triloop.cli browse sim.h5
```

For the end-to-end “does the alias arithmetic come out right” check on a synthetic 4-channel 15.625 MS/s capture, see `triloop/triloop/bands.py` (specifically `extract_bands` will raise `ValueError` if the requested per-band bandwidth overlaps the Nyquist edge or another band’s alias, which is the nearest thing to an automatic correctness check).